

Deep Learning

Resumo Preparatório para Avaliação

Compilação dos Módulos T1 a T4

Tópico 1: Redes Neurais Artificiais (FFNN)

1. Conceitos Básicos e Analogia Biológica

As Redes Neurais Artificiais (RNA) são modelos de *Machine Learning* (Aprendizagem Automática) inspirados no funcionamento do cérebro humano. O cérebro supera a lentidão das suas unidades individuais através de uma vasta estrutura paralela e não-linear.

- Numa RNA, o **conhecimento é armazenado nas ligações** entre os neurónios.
- O processo de aprendizagem (treino) ajusta sistematicamente os "pesos" (**w**) destas ligações utilizando dados recolhidos do ambiente.

2. O Neurónio Artificial e Funções de Ativação

A unidade básica do modelo é o neurónio artificial. Cada neurónio executa dois passos matemáticos principais:

1. **Soma Ponderada:** Multiplica cada input (**x_i**) pelo seu peso correspondente (**w_i**), somando os resultados e adicionando um *bias* (**b**). Equação: $z = \sum w_i x_i + b$.
2. **Função de Ativação:** O resultado **z** é passado por uma função de ativação para introduzir não-linearidade e limitar/determinar a amplitude do sinal de saída (**y = f(z)**).

PRINCIPAIS FUNÇÕES DE ATIVAÇÃO:

- **Sigmoid / Logística:** Converte valores para o intervalo (0, 1). Tradicionalmente muito usada para probabilidades, embora sofra do problema de *vanishing gradients*.
- **Tanh:** Semelhante à sigmoid, mas o intervalo é (-1, 1).
- **ReLU (Rectified Linear Unit):** Função que devolve **0** para valores negativos e o próprio valor (**y = x**) para valores positivos. Atualmente é o padrão na grande maioria das Deep Neural Networks por acelerar a convergência do treino.
- **Linear:** Usada nos nós de output para problemas de regressão.

3. Arquitetura e Topologia da Rede

As **Feedforward Neural Networks** (também conhecidas como Multilayer Perceptrons - MLPs) estruturam os neurónios em camadas, onde o sinal avança num sentido único:

- **Input Layer:** Recebe as *features* dos dados. Os dados são frequentemente padronizados previamente.
- **Hidden Layers (Camadas Ocultas):** Processam as abstrações; é aqui que reside a profundidade da rede (Deep Learning).
- **Output Layer:** Produz a predição final. A configuração depende do tipo de problema.

Configurações Típicas da Output Layer:

Tipo de Problema	Nós de Output	Função de Ativação	Loss Function (Custo)
Classificação Binária	1	Sigmoid	Binary Cross-Entropy
Classificação Multiclasse	1 por classe (One-hot)	Softmax	Categorical Cross-Entropy
Regressão	1	Linear (ou ReLU)	Mean Squared Error (MSE)

4. O Algoritmo de Treino: Backpropagation

O objetivo do treino supervisionado é descobrir a configuração de pesos e biases que minimiza uma função de custo (*Loss function*). Esse processo é essencialmente baseado em derivadas.

- **Forward Propagation:** O input viaja pela rede até gerar a predição e, consequentemente, calcular o erro na função de custo.
- **Backward Propagation:** O erro é propagado de volta para as camadas anteriores. A **Chain Rule** (regra da cadeia de funções compostas) é usada no "computation graph" para calcular a derivada do erro em relação a cada peso.
- **Gradient Descent e Learning Rate:** Uma vez calculado o gradiente, os pesos são atualizados na direção descendente da superfície do erro. O *Learning Rate* dita a amplitude de cada passo. O processo repete-se por vários "epochs" ou iterações.

Tópico 2: Deep Learning (DL) e CNNs

1. O que é Deep Learning?

O **Deep Learning** é um subcampo do Machine Learning caracterizado pela utilização de redes com elevada complexidade computacional e muitas camadas (profundidade). Isto permite às redes criar modelos matemáticos internos que extraem características (*features*) de forma progressivamente abstrata (ex: identificar desde simples píxeis e linhas até formas inteiras, como rodas, e depois "carros").

O sucesso do DL hoje deve-se a enormes volumes de dados estruturados (big data) e saltos exponenciais no processamento paralelo por GPUs (Placas Gráficas).

2. Otimização e Melhorias nos Algoritmos de Treino

Com o crescimento da profundidade e complexidade das redes, surgiram inovações para otimizar os métodos de aprendizagem tradicionais:

- **Mini-Batches:** Em vez de usar todo o dataset para calcular uma única atualização de peso, os dados são agrupados (ex: lotes de 64, 256 imagens) que cabem inteiros na memória da GPU. Quando o tamanho do batch é 1, falamos de "Stochastic Gradient Descent".
- **Momentum:** Uma média móvel dos gradientes recentes que adiciona inércia, acelerando a direção correta e reduzindo saltos abruptos ("update noise").
- **RMSProp e Adam:** O Adam (Adaptive Moment Estimation) combina Momentum e RMSProp, adaptando as taxas de aprendizagem de forma individual para cada peso baseado na magnitude e variação recente dos gradientes.

3. Overfitting e Generalização

Um modelo muito complexo (muitos pesos) pode facilmente memorizar os dados de treino (overfitting), perdendo a capacidade de generalização para novos casos.

- **Early Stopping:** O erro num dataset separado ("validação") é monitorizado. Se parar de decrescer por algumas épocas consecutivas, o treino para preventivamente.
- **Dropout:** Uma técnica exclusiva ao DL onde, a cada iteração de treino, uma percentagem aleatória dos neurónios é desativada artificialmente (colocada a zero), forçando os neurónios restantes a aprender em equipa e a não confiar numa só característica específica.

4. Convolutional Neural Networks (CNNs)

As CNNs foram a evolução ideal para tratar dados em grelha, especialmente imagens ou dados espaciais (Tensores 3D: altura $\times$ largura $\times$ profundidade de canais RGB).

- **Camadas Convolucionais:** Utilizam "filtros" ou "kernels" – pequenas matrizes matemáticas (ex: 3x3) que deslizam pela imagem aplicando multiplicações ponto-a-ponto, efetuando o detetar de bordas, contrastes ou texturas independentemente da posição destas na imagem (Invariância Translacional).
- **Padding & Stride:** *Stride* define o passo/salto da janela de convolução ($\text{stride} > 1$ reduz dimensionalidade). O *Padding* adiciona bordas neutras de píxeis de valor zero para preservar o tamanho original da imagem durante as convoluções.
- **Pooling Layers:** Reduzem a dimensionalidade das feature maps. O método mais comum é o Max Pooling (fica apenas com a resposta mais forte numa área).
- A arquitetura padrão consiste em repetições de [Convolução -> ReLU -> Pooling] culminando num "Flatten" para ligar a camadas Densas (MLP) clássicas para a predição final.

Dicas para Lidar com Poucos Dados (CNNs):

1. **Data Augmentation:** Gerar mais amostras aplicando transformações (zoom, rotação, recorte, espelhamento) nas imagens de treino existentes.
2. **Transfer Learning:** Importar as camadas convolucionais de um modelo pré-treinado em enormes bases de dados externas. "Congelar" (freeze) os seus pesos (para manter a aprendizagem de abstrações gerais) e substituir/treinar apenas a cabeça de classificação final.

5. Processamento de Sequências

Séries temporais ou textos requerem topologias especializadas, como as Recurrent Neural Networks (RNNs).

- **Pré-Processamento de Texto (NLP):** Como os modelos necessitam de matemática, textos sofrem Tokenização (divisão em palavras ou subpalavras) e Codificação Numérica (ex: One-hot encoding ou **Word Embeddings**). Os embeddings mapeiam relações semânticas para um espaço vetorial denso e compacto.
- **RNN e LSTMs:** As RNNs contêm laços (loops) que mantêm o estado oculto de passos anteriores da sequência. Para resolver a dificuldade de memorizar relações longas ("vanishing gradient" através do tempo), foram criadas as **Long Short-Term Memory (LSTM)**. As LSTMs introduzem "gates" (Portas de Input, Forget e Output) que decidem intencionalmente qual a informação antiga a guardar e qual a esquecer.

Tópico 3: Transformers e LLMs

1. O Mecanismo de Atenção e a Arquitetura Transformer

A arquitetura "Transformer", descrita em "Attention is All You Need" (2017), abandonou a dependência sequencial típica das RNNs/LSTMs em favor de puro paralelismo.

- **Self-Attention:** Avalia a relevância contextual partilhada entre cada par de palavras de uma frase e traduz isso num vetor representativo e dependente desse contexto. O processo assenta na multiplicação linear de 3 matrizes derivadas do input: **Query (Q)**, **Key (K)** e **Value (V)**.
- **Multi-Head Attention:** Usa a self-attention através de várias projeções de atenção simultâneas e paralelas (as "heads"). Assim, pode isoladamente analisar relações sintáticas e relações semânticas na mesma passagem.
- **Positional Encoding:** Dado que o Transformer não lê palavras em sequência (processa todas ao mesmo tempo para máxima eficiência paralela), ele adiciona uma equação posicional fixa ao *embedding* inicial de cada token, indicando a sua ordem na frase original.
- **Encoder vs. Decoder:** A arquitetura clássica mapeia de sequência para espaço latente (Encoder) e deste para nova sequência (Decoder) — ideal para tradução e sumarização.

2. A Era dos LLMs (Large Language Models)

Modelos colossais baseados em subcomponentes do Transformer deram origem ao Boom GenAI recente.

- **BERT (Bidirectional Encoder):** Desenvolvido pela Google (2018), utiliza *apenas Encoders* para aprender de forma bidirecional. É treinado com estratégias nativas de *Masked Language Modeling* (tentar prever palavras tapadas a meio da frase) e *Next Sentence Prediction*. Otimizado para classificação e extração semântica.
- **GPT (Generative Pre-trained Transformer):** Baseado em *apenas Decoders Causais*, ou seja, gera de forma sequencial prevendo a probabilidade matemática do próximo *token* sem poder espreitar o "futuro". Otimizado para geração fluída a partir de **Prompts**.

3. Afinação (Fine-Tuning) e Alinhamento de LLMs

Transformar um gigante modelo base de conhecimento numa "ferramenta/assistente de chat" funcional exige passos intermédios.

- **Reinforcement Learning from Human Feedback (RLHF):** Um método crucial na segurança/usabilidade da GenAI atual. Modelos pre-treinados são avaliados empiricamente por humanos que classificam as melhores respostas dadas a conversas; cria-se um *Reward Model* (Modelo de

Recompensa) a partir dos seus votos que posteriormente afina o comportamento do LLM original utilizando algoritmos como o PPO (Proximal Policy Optimization).

- **LoRA (Low-Rank Adaptation):** O PEFT (Parameter-Efficient Fine-Tuning) resolve as elevadas exigências de RAM no ajuste de Modelos Maciços, não atualizando todos os milhares de milhões de pesos. No caso do LoRA, insere matrizes de baixa ordem nas camadas de atenção enquanto que os pesos mestres congelam.

4. Casos de Uso Avançados: RAG, Agentes e Prompt Engineering

A engenharia dos Prompts visa controlar a estocasticidade do modelo (parâmetro **Temperature** aumenta/baixa a criatividade) para resultados úteis e precisos.

- **Zero-shot / Few-shot Learning:** Graças aos extensos treinos com instruções, o LLM resolve problemas nunca antes vistos pelo simples facto do utilizador lhe dar comandos textuais precisos, por vezes sendo auxiliado com 1 a 5 exemplos na mensagem de **prompt** (Few-shot learning).
- **Retrieval-Augmented Generation (RAG):** Sistema fidedigno (low hallucination) onde o prompt de pesquisa é antes convertido para embedding e busca documentos de uma base de dados externa; a resposta foca-se obrigatoriamente neste conteúdo externo acedido (os referidos "chunks").
- **Agent Frameworks:** O LLM planeia operações de múltiplos passos usando a ReAct (Reasoning and Acting) e invocando "Tools" (ferramentas Web, interpretadores) ou interagindo num modelo Multi-Agente (Autogen).

Tópico 4: Generative Models e Reinforcement Learning

1. Modelos Generativos

O Machine Learning é maioritariamente preditivo/discriminativo (reconhece cães e gatos). Os Modelos Generativos têm como foco modelar estatisticamente os dados subjacentes originais, permitindo a criação livre de novos dados credíveis que não existiam.

- **Autoencoders:** Modelos com estrutura em funil. Um *Encoder* extrai características forçando-as num "espaço latente" vetorial condensado (o funil/gargalo), e um *Decoder* tenta recriar a imagem ou texto originais perfeitamente a partir de lá. Se sofre de overfit o modelo torna-se rígido.
- **Variational Autoencoders (VAEs):** Solucionam a rigidez exigindo que o Encoder mapeie dados para parâmetros de distribuição normal estocástica (Média e Variância). O Decoder gera novos *outputs* baseando-se em amostras aleatórias recolhidas dessas distribuições regularizadas, criando uma geração interpolada contínua.
- **GANs (Generative Adversarial Networks):** Envolvem o treino cruzado ("jogo competitivo") de duas redes adversárias simultaneamente. O "Gerador" cria *fakes*, e o "Discriminador" avalia imagens e tenta decifrar a original e as criadas. A loss function cruza resultados empurrando o Gerador a tornar as suas criações idênticas ao real. Muito difíceis de estabilizar (convergir).

Estudo de Caso Prático: Projeto Shikifactory100

O objetivo do consórcio consistia em desenhar (em computador), validar experimentalmente e sintetizar de raiz moléculas com características não-tóxicas que imitam a capacidade de adocicar da Biologia. Utilizando Autoencoders profundos em grandes datasets moleculares da Base de dados "PubChem", o algoritmo explorou quimicamente o espaço latente para prever propriedades teóricas organolépticas antes dos ensaios em laboratório, provando a versatilidade de AI num design racional de Biofábricas Químicas.

2. Fundamentos de Reinforcement Learning (Aprendizagem por Reforço)

Muda-se do cenário onde temos "Labels e Input Data", focando a formulação em métodos teóricos de controlo reativo contínuo, aplicável à Robótica ou Gestão de Sistemas Autónomos e Financeiros.

- **Elementos Chave:** A arquitetura RL assenta num **Agente** inserido num **Ambiente**, tomando **Ações** dinâmicas ao aperceber-se do seu **Estado** e procurando atingir um objetivo a longo prazo.
- **Markov Decision Processes (MDP):** Modelo estocástico clássico com os componentes: Estados finitos (**S**), Ações viáveis (**A**), Recompensas (**R**) e dinâmica de transição (**P**). Incorporam ainda o parâmetro *Gamma* para descontar recompensas diferidas ou atrasadas do futuro vs as instantâneas.
- **Exploration vs. Exploitation:** Grande desafio do RL: se o Agente apenas executa as estratégias das quais obteve retorno já demonstrado (Exploitation), pode ficar encurralado na minimização não-ideal e ignorar novos trajetos onde as Ações iniciais podiam ter pontuação nula na recompensa original, mas que desencadeiam melhores lucros no culminar (Exploration).
- **Q-Learning (Value-based):** Aprende empiricamente iterando e medindo a melhor função e Qualidade (Q) da recompensa agregada obtida se escolhermos a "Ação - X" naquele cenário ou num "Estado - Y", baseando-se em tabelas e equações base de Bellman.

3. Deep Reinforcement Learning (Deep RL)

Usa a versatilidade poderosa das Redes Profundas em cima de RL.

- **DQN (Deep Q-Networks):** Um método introduzido pela DeepMind que ensinou com sucesso a AI a ganhar clássicos videojogos arcade (Atari) baseando as observações de "estado" unicamente nos frames pixelizados visuais, aplicando para tal CNNs para digerir as imagens emparelhadas à Reward (Pontuação/Score do próprio ecrã).
- **Exemplos Históricos de Sucesso em Deep RL:** Sistemas como o AlphaGo (que domina perfeitamente o Go), AlphaFold (mapeamento universal e dobramento da biologia proteica tridimensional) e mesmo resolução heurística de labirintos/cubos mágicos.

Documento gerado automaticamente pela Inteligência Artificial Gemini (Google).

Materiais de referência e extração via: T1-FFNN.pdf, T2-DL.pdf, T3-TransfLLM.pdf, T4-GenModels.pdf.